Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Elektro- und Informationstechnik

Bachelorarbeit

von

Thomas **Mustermann**

LaTeX im Griff

LaTeX under control

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Elektro- und Informationstechnik

Bachelorarbeit

von

Thomas **Mustermann**

LaTeX im Griff

LaTeX under control

Bearbeitungszeitraum:	von	1. Oktober 2013
	bis	31. Januar 2014

1. Prüfer: Prof. Dr. Axel Maier

2. Prüfer: Prof. Dr. Bruno Müller

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Eigenständigkeitserklärung gemäß § 27 (8) ASPO

Name und Vorname
der Studentin/des Studenten: **Mustermann, Thomas**

Studiengang: **Elektro- und Informationstechnik**

Ich bestätige, dass ich die Bachelorarbeit mit dem Titel:

LaTeX im Griff

selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Datum: May 3, 2026

Unterschrift:

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Bachelorarbeit Zusammenfassung

Studentin/Student (Name, Vorname):	**Mustermann, Thomas**
Studiengang:	Elektro- und Informationstechnik
Aufgabensteller, Professor:	Prof. Dr. Axel Maier
Durchgeführt in (Firma/Behörde/Hochschule):	Renner-Verlag, Musterstadt
Betreuer in Firma/Behörde:	Dr. Karl Schulze
Ausgabedatum: 1. Oktober 2013	Abgabedatum: 31. Januar 2014

Titel:

LaTeX im Griff

Zusammenfassung:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Schlüsselwörter: LaTeX, Textsatz, Formeln, Graphiken

Contents

1 Introduction **1**
1.1 Motivation 1
1.2 Project Goal 2
1.3 Research Focus – Imitation Learning 3

2 Basics Classical Robotics **4**
2.1 Erster Abschnitt von Kapitel 2 4
2.1.1 Ein Unterabschnitt 4
2.1.1.1 Ein Unterabschnitt 4

3 Basics Intelligent Robotics **5**
3.1 Machine Learning Basics 5
3.1.1 Mathematical Notation of formulas 5
3.1.2 Glossary 5
3.1.3 Core Machine Learning Categories 6
3.2 Intelligent robots with Deep Learning (RL) 7
3.2.1 Reinforcement Learning 8
3.2.1.1 Mathematical description of the RL problem: 9
3.2.1.2 Knowledge Modality 11
3.2.2 Imitation Learning 12
3.2.2.1 Behavior Cloning (BC) 13
3.2.2.2 Learning from Videos / Observation 13

4 Imitation Learning fundamentals **15**
4.1 The Data Problem in Robotics 15
4.2 Why Imitation Learning? 15
4.2.1 Idea behind Imitation Learning: 15
4.2.2 Instruct robots 16

5 5 **20**

Literaturverzeichnis **21**

Abbildungsverzeichnis **23**

Tabellenverzeichnis **24**

Symbole, Formelzeichen und Einheiten

Chapter 1

Introduction

1.1 Motivation

Roboter werden zukünftig eine Schlüsselrolle in zahlreichen Anwendungsfeldern einnehmen. Be-reits heute steigt der Einsatz von Robotersystemen in der industriellen Produktion kontinuierlich an, da diese insbesondere monotone, einfache und repetitive Aufgaben effizient und zuverlässig übernehmen können. Trotz der teilweisen sehr hohen Anschaffungskosten, investieren immer mehr Unternehmen in die Integration von Roboter in Ihre Produktionslinien. Die Vorteile liegen dabei vor allem in der hohen Verfügbarkeit und Planbarkeit: Nach der Inbe-triebnahme fallen nur noch geringe laufende Betriebskosten an, die sich primär auf Energiever-brauch in From von Strom- und Wartungskosten beschränken. Darüber hinaus können Roboter unabhängig von der Arbeitszeit 24h, 7 Tage die Woche betrieben werden und liefern eine gleich-bleibende Qualität. Dies ermöglicht eine hohe Prozessstabilität sowie eine signifikante Steigerung der Produktivität. Im Gegensatz zu menschlichen Arbeitskräften, werden Roboter nicht krank, nehmen nicht an einen Gewerkschaftsstreik teil und es müssen auch keine Sonderzahlungen in Form von Nacht-, Wochenend- und Feiertagszuschlägen geleistet werden. Grundsätzlich lassen sich Robotersysteme hinsichtlich ihrer Programmierung in zwei Kategorien einteilen. In der industriellen Praxis dominieren derzeit statisch programmierte Roboter, die für klar definierte Aufgaben in strukturierten Umgebungen eingesetzt werden. Sie haben feste Bewe-gungsabläufe und können sich nur in geringem Maße mithilfe integrierter intelligenter Sensoren an ihre Umgebung bzw. an die jeweilige Aufgabe anpassen. Solche Systeme sind besonders effi-zient in standardisierten Produktionsprozessen, stoßen jedoch an ihre Grenzen, sobald sich die Umgebung oder die Anforderungen ändern. Im Gegensatz dazu gewinnen intelligente Robotersysteme zunehmend an Bedeutung. Diese sind in der Lage, auf Veränderungen in ihrer Umgebung zu reagieren und ihr Verhalten adaptiv anzu-passen. Insbesondere durch den Einsatz von Künstlicher Intelligenz eröffnen sich neue Möglich-keiten, komplexe und unstrukturierte Aufgaben zu bewältigen. Zentrale Herausforderungen bestehen jedoch hierdabei in der robusten Ausführung vielfältiger Szenarien sowie in der Fähigkeit, auf neue Objekte, Umgebungen und Aufgaben zu generalisieren. Die heute eingesetzten intelligenten Robotersysteme, bzw.

deren zugrunde liegenden KI-Modelle sind oftmals stark auf ihre jeweilige Aufgabe und die Zielumgebung spezialisiert, bzw. trainiert worden. Aufgaben, die nicht im Trainingsdatensatz enthalten sind, können häufig nicht zuverlässig ausge-führt werden. Ebenso treten Probleme auf, wenn gelernte Aufgaben in grundlegend verschiedenen Umgebungen ausgeführt werden sollen. Der Grund hierfür liegt in der Art und Weise, wie die zugrunde liegenden KI-Modelle trainiert werden: Die Trainingsdaten sind in der Regel stark auf eine spezifische Aufgabe und Umgebung zugeschnitten. Eine Erhöhung der Datenvielfalt führt dabei häufig zu einem Zielkonflikt, da sie mit einer geringeren Genauigkeit bei der Ausführung einzelner Aufgaben einhergehen kann. Selbst wenn man ein großes generalisierendes Roboter KI-Modell entwickeln möchte, stellt der Mangel an geeigneten und vielfältigen Roboter-Trainingsdaten ein gravierendes Problem dar. Klassische, aufgabenspezifische Trainingsansätze stoßen hierbei aufgrund mangelnder Skalierbar-keit und Flexibilität an ihre Grenzen. Aktuelle Forschungsansätze verfolgen daher das Ziel, Robo-tern ein visuelles und semantisches Aufgabenverständnis zu vermitteln, sodass sie Handlungen aus hochdimensionalen Eingaben wie Videos oder Bildern ableiten können. Dadurch können neue Datenquellen für das Training des Roboters herangezogen werden. Mit einem visuellen und se-mantischen Verständnis können beispielsweise (menschliche) Videos von YouTube im Training verwendet werden. Diese beinhalten viele verschiedene Handlungen und Ausführungsformen ver-schiedenster Aufgaben, die ein generalisierendes Roboter KI-Modell brauchen würde. Imitation Learning ist ein vielversprechender Ansatz in diesem Kontext, der in dieser Arbeit genauer unter-sucht wird. Beim Imitation Learning geht es darum, den Roboter seine Aufgabe mithilfe eines (menschlichen) Demonstrationsvideos beizubringen (z.B. das Stapeln von Würfeln, oder das Greifen von Objek-ten). Beim traditionellen Ansatz muss ein Roboter auf die jeweilige Aufgabe mit viel Aufwand bzw. mit vielen Roboter trajectory Datensätzen trainiert werden.

1.2 Project Goal

Ziel dieser Arbeit ist es eine Einführung in das Thema Robotic mit Imitation Learning zu liefern. Die Arbeit gliedert sich in zwei Teile. Dabei werden zuerst die Grundlagen der Robotik erläutert und anschließend technisch in das Thema Intelligente Robotik mit dem Fokus Imitation Learning eingeführt. Abgerundet wird der erste Teil mit einem Literaturüberblick, über ausgewählte Imitation Learning Ansätze, die dort näher untersucht werden. Im zweiten Teil dieser Arbeit wird ein Open-Source Imitation Learning Modell experimentell auf einen realen Roboterarm implementiert. Die Erstmalige Inbetriebnahme des Roboters und das Setup, wie ein KI-Modell zur Ausführung auf den Roboter gebracht werden kann, gehört auch dazu. Ziel ist es, den Roboter zu befähigen, einfache Manipulationsaufgaben, beispielsweise das Stapeln von Würfeln, allein auf Basis menschlicher Demonstrationsvideos zu erlernen und auszu-führen. Die Leistungsfähigkeit des vorgeschlagenen Ansatzes wird anschließend experimentell auf dem realen Robotersystem untersucht und evaluiert.

Beim Imitation Learning geht es darum, den Roboter seine Aufgabe mithilfe eines (menschlichen) Demonstrationsvideos beizubringen (z.B. das Stapeln von Würfeln,

oder das Greifen von Objek-ten). Beim traditionellen Ansatz muss ein Roboter auf die jeweilige Aufgabe mit viel Aufwand bzw. mit vielen Roboter trajectory Datensätzen trainiert werden. Beim Imitation Learning geht es darum, den Roboter seine Aufgabe mithilfe eines (menschlichen) Demonstrationsvideos beizubringen. Dabei nimmt eine Kamera ein Video von der jeweiligen Person auf, die eine bestimmte Aufgabe ausübt. Anschließend wird dem Roboter-KI-Modell das Demonstrationsvideo der Aufgabe gezeigt, welches wiederum den Roboter entsprechend steuert, um die jeweilige Aufgabe in seiner Umgebung auszuführen.

1.3 Research Focus – Imitation Learning

Chapter 2

Basics Classical Robotics

2.1 Erster Abschnitt von Kapitel 2

test test test test

2.1.1 Ein Unterabschnitt

2.1.1.1 Ein Unterabschnitt

jetzt gehts

Chapter 3

Basics Intelligent Robotics

3.1 Machine Learning Basics

3.1.1 Mathematical Notation of formulas

Capital letters are used for random variables, e.g. S represents the set of all states in which the robot can be and A represents the set of all possible actions it can perform.

Lowercase letters are used for specific values of these random variables, e.g. s stands for a specific robot state and a for a specific action that the robot performs. A lowercase letter is usually followed by a time index $_t$, representing the specific action taken by the robot at time step t. In some cases the time index is omitted for simplicity, e.g. a instead of a_t and the next state or action is then denoted as s' or a' instead of s_{t+1} or a_{t+1}.

Capital letters in bold represents Matrices, e.g. the matrix $\mathbf{X}$.

Lowercase letters in bold are used for specific values of (random) variables that are vectors, e.g. $\mathbf{x}$ represents a specific vector.

3.1.2 Glossary

Agent & Policy:

An agent is an entity that learns to perform a specific behavior through interaction with its environment. In the context of this work, the agent corresponds to the robot, which has to execute a given action or task. The policy can be interpreted as the brain of the agent, determining which action to take next in a given situation in order to achieve the task objective. In our example, the policy controls the robot's motion and actions.

Formally, a policy can be represented as

$$\pi_\theta(a \mid s) \tag{3.1}$$

which defines a probability distribution over actions a, conditioned on the current state s. In other words, the policy specifies how likely the agent is to select a particular action given its current observation of the environment.

In the classical meaning, the action a corresponds for example to a robot command, such as moving the gripper to a target position (x, y, z) or setting a joint angle to a specific value with a defined velocity. The state s represents the agent's observation, which may include, for example, a sequence of recent camera images (e.g., the last four frames) as well as additional sensor inputs such as proprioceptive signals.

Robot Trajectory:

A robot trajectory describes how the robot moves from a starting state to a target state, including position, orientation and, if applicable, speeds, accelerations, and forces acting on the robot. A robot tracetory is a temporal sequence of actions and the resulting robot states.

Annotation:

Annotation refers to the process of labeling data. Data is provided with additional information. For example, class affiliations can be determined, target objects can be marked in images, or states and actions can be marked in robot trajectories. Annotated data, or labeled data, form the basis for supervised learning approaches.

Annotating refers to the process of labeling data, in which data is enriched with additional information. For example, this may include assigning class labels, marking target objects in images, or labeling states and actions within robot trajectories. Annotated, or labeled, data forms the foundation for supervised learning approaches.

3.1.3 Core Machine Learning Categories

There are various approaches to training an AI model for a robot. These involve methods from the four core machine learning categories, which can also be combined with one another:

Supervised Learning:

In supervised learning, a model is trained using labeled training data. This means that, for the input data, the output data to be predicted are known. The goal is to train a model that correctly predicts the output data and makes reliable predictions on unseen data.

Unsupervised Learning:

In contrast to supervised learning, unsupervised learning uses unlabeled data for model training. The model tries to recognize structures, patterns or relationships in the data on its own.

Semi Supervised Learning:

Semi-supervised learning is a combination of supervised and unsupervised learning. In this approach, a model is trained using a small amount of labeled data and a large amount of unlabeled data. The goal is to leverage the advantages of both approaches, particularly when annotating data is time-consuming or costl-intensive. This is relevant in robotics, as the data used in that field is often difficult to label.

Reinforcement Learning:

Reinforcement learning (RL) differs fundamentally from the previously mentioned approaches. Instead of learning from fixed datasets, an agent actively interacts with its environment and tries to find out an optimal strategy to reach a defined goal. Through its actions, it influences the state of the environment and subsequently receives feedback in the form of rewards or penalties, see Figure 3.1 for a short overview. In the context of robotics, the robot is the agent. The objective is to learn a strategy (= policy) that maximizes the cumulative reward over time for the given task.

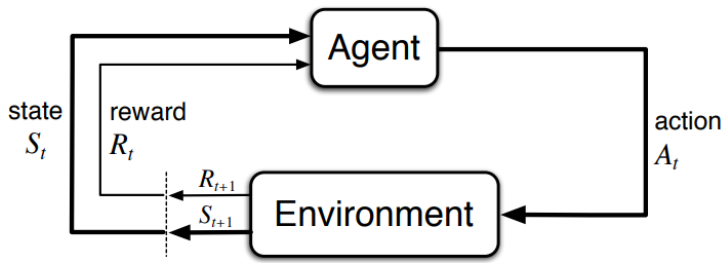


Figure 3.1: Base concept of Reinforcement Learning

The agent proceeds in its environment according to the trial-and-error principle to learn an optimal policy. Safety is a critical concern during training, as the agent may execute unpredictable actions while exploring the state space. For this reason, simulation environments are a central component of training and are frequently used for pre-training the policy. The policy is then subsequently often fine-tuned in the real-world environment. Because the agent learns through trial and error, more efficient, unknown, and creative solutions can be found that a human might not have come up with. See section 3.2.1 for more details.

3.2 Intelligent robots with Deep Learning (RL)

In general, modern robot training distinguishes between two main approaches: imitation learning and reinforcement learning. Each approach uses a different strategy for training the robot model. In Figure 3.2 an overview of the two approaches is given. The following sections explain the two approaches in more detail and provide a mathematical description of the underlying problem.

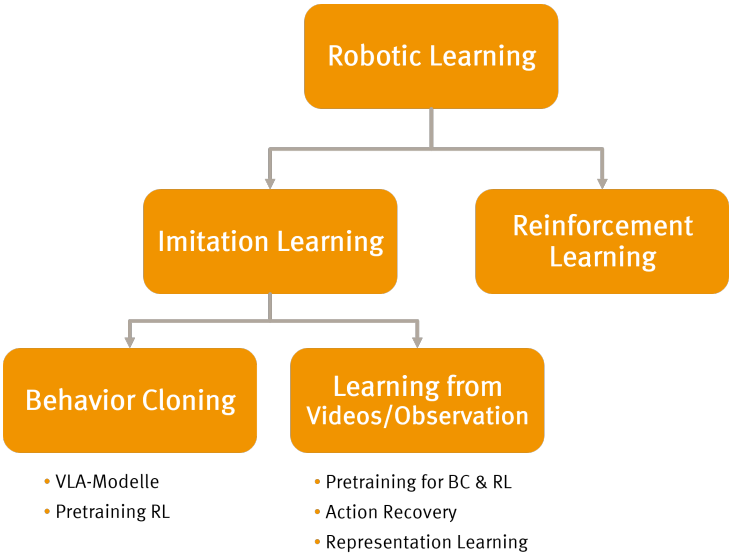


Figure 3.2: Overview Robotic Learning

3.2.1 Reinforcement Learning

Reinforcement learning (RL) is both a fundamental discipline of machine learning and a distinct approach within Robot Learning. The robot learns a policy through trial and error using rewards provided from the given task and the environment. The actions performed by the agent are evaluated with a reward score, and this score is then used to gradually optimize the policy.

Key components of the RL problem The following section explains the key concepts in RL related to robotics, see also figure 3.1 for a general graphical overview.

- **Agent:** The agent is the entity that learns to perform a specific behavior through interaction with its environment. In the context of this work, the agent corresponds to the robot, which has to execute a given action or task.
- **Environment:** The environment is the setting in which the agent (the robot) operates. Simply put, the environment is the robot's workplace, where it is deployed. In addition to the robot itself, it includes all relevant objects with which the robot interacts. The environment can be either the real world or a simulated environment in which the robot is trained.
- **Robot Actions:** The Robot actions are simply the control commands the robot can send to its actuators, e.g. open or close the gripper, move the arm to position X, Y, Z, or the joint angles of the individual robot axes, etc.
- **Reward:** The reward is a numerical value that indicates how well the robot is performing in relation to the task objective. For example, if the robot successfully picks up an object, it might receive a positive reward, while if it fails or drops the object, it might receive a negative reward. The goal of Reinforcement Learning is to learn a policy that maximizes the cumulative reward over time.
- **Robot State:** state is a complete description of the situation in the environment

at a particular point in time. The robot state represents the current situation of the robot and its environment. It can include various types of information, such as the robot's position, orientation, joint angles, sensor readings, and even visual input from cameras [14]. The state is crucial for the robot to make informed decisions about which actions to take.

- **Observation:** An observation is the information that the robot receives from its sensors at a given time step. It may not provide a complete picture of the environment, and thus may not fully represent the true robot state. In our case, the observation is mostly equivalent to the state [14].
- **Policy:** The policy can be interpreted as the brain of the agent. The policy determines which action should be taken next when the agent is in a particular state in order to achieve the task objective. The Policy controls the robot's motion and actions.

3.2.1.1 Mathematical description of the RL problem:

Die RL Modularitäten werden wie folgt dargestellt:

- Action: a
- Reward: r
- State: s
- Observation: o
- Policy: π
- Time step: t, e.g. a_t is the action taken at time step t, a_{t+1} is the action taken at the next time step, etc.
- s': The next state after taking action a in state s, i.e. $s' = s_{t+1}$, $s = s_t$.

<u>Markov Decision Process</u>

Reinforcement Learning is based on the Markov decision process (MDP). A MDP is a stochastic decision-making process that formally models the transition probabilities from a state s to a subsequent state s' provided that action a is performed, expressed mathematically as $P(s'|s,a)$ [12, 8].

The objective is to learn a policy $\pi(a|s)$ that maximizes the expected long-term cumulative reward, defined as $\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$. The discount factor $\gamma \in [0,1]$ determines the importance of future rewards compared to immediate rewards. A value of γ close to 0 makes the agent focus more on immediate rewards, while a value close to 1 encourages the agent to consider long-term rewards more heavily [12, 4].

An MDP also defines a reward function that specifies the reward r an agent receives when taking action a in state s and transitioning to a subsequent state s'. This is commonly expressed as $R_a(s,s')$ [12, 8].

The **Markov property** says that all information from the past is contained in the current state, mathematically expressed as $P(S_{t+1}|S_t) = P(S_{t+1}|S_1, S_2, ...S_t)$. In other words, the transition probabilities depend only on the current state and not on the history of previous states [4].

In practice, the Markov property is often not strictly satisfied. For example, a single image captured by a camera does not provide sufficient information to infer the dynamics of the environment. From one image alone, it is not possible to determine whether a car is moving forward at 100 km/h or 10 km/h, reversing, or remaining stationary. However, such information is crucial for making correct decisions in applications like autonomous driving [11]. Therefore, in practice, only sufficiently informative state representations are often used because the ideal Markov property is not strictly fulfilled. This is often achieved by incorporating a history of observations, such as a sequence of the last five camera frames, to better capture the underlying dynamics of the environment [8].

Training process:

Zu Beginn des Trainingsprozesses ist die Policy in der Regel zufällig, was bedeutet, dass der Roboter zunächst zufällige Bewegungen ausführt. Durch Ausprobieren versucht der Roboter nun Aktionen zu finden, die mit einer hohen Belohnung einhergehen. Diese Aktionen werden anschließend von der Policy höher gewichtet, was bedeutet, dass der Roboter in Zukunft eher dazu neigt, diese Aktionen auszuführen. Für eine erfolgreiche Lernphase ist es wichtig, dass der Roboter eine gute Balance zwischen Exploration (Ausprobieren neuer Aktionen im jeweiligen Zustand) und Exploitation (Nutzung bereits bekannter, erfolgreicher Aktionen im jeweiligen Zustand) findet. Auf die Exploration wird zu Beginn des Trainingsprozesses mehr Wert gelegt, um die Umgebung besser kennenzulernen und die effizientesten Aktionen im jeweiligen Roboter State für den Erfolg der Aufgabe zu finden. Im Laufe der Zeit, wenn der Roboter mehr über die Umgebung und die Belohnungsstruktur gelernt hat, wird die Exploitation wichtiger, um die bereits gefundenen erfolgreichen Aktionen häufiger auszuführen und so die Gesamtleistung, die am Aufgabenerfolg gemessen wird zu verbessern. Es existieren zahlreiche Methoden und Algorithmen, wie die Exploration und Exploitation Phasen effektiv gesteuert werden können. Beispielalgorithmen sind ϵ-greedy, Upper Confidence Bound (UCB), Thompson Sampling. Im Rahmen der Arbeit wird aber nicht näher darauf eingegangen.

Durch die Explorationsphase wird erst spät der Erfolg des gewählten RL-Ansatzes sichtbar, anders als beim Klassischen Deep Learning, siehe Abbildung 3.3.

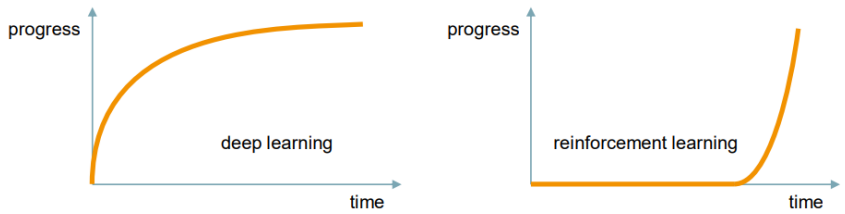


Figure 3.3: Training Progress in Deep Learning and Reinforcement Learning

3.2.1.2 Knowledge Modality

An RL Knowledge Modality (KM) is some function learned from data that represents specific types of RL-related knowledge. KMs can be learned through trial-and-error principles or from various data sources, like video datasets, and can capture different aspects of the RL problem.

Knowledge Modalities (KMs) from Video

- `Policy:` A Policy $\pi(a_t|s_t)$ is a mapping from states to actions and defines the behavior of the agent that is in a particular state. $\pi(a_t|s_t)$ is the probability of taking action a_t given that the agent is in state s_t. $_t$ denotes the time step. In other words, the policy specifies how likely the agent is to select a particular action given its current observation of the environment. It can be learned from video data by observing the actions taken in different states and learning to predict those actions based on the visual input.
- `Value functions:` Value functions estimate the expected future return (cumulative reward) for a given state or state-action pair. They can be learned from video data by observing the outcomes of actions and learning to predict the expected future rewards based on the visual input.
  - `State-value function (V-function):` A V-function $V_\pi(s_t)$ is a mapping from states to the expected future return when acting under a particular policy π.

    Intuition: "How good is it to be a certain state, if I follow my policy π?"

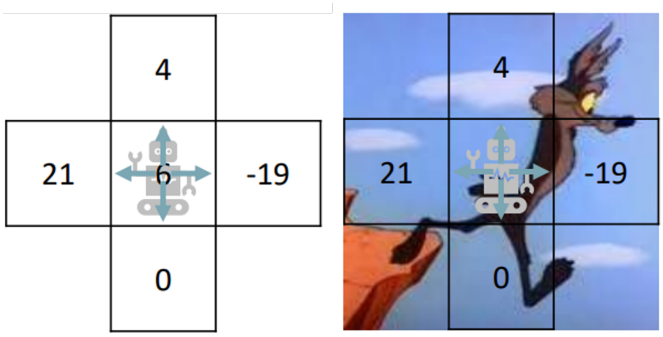


Figure 3.4: State-Value Function

  - `State-action value function (Q-function):`

    A Q-function $Q_\pi(s_t, a_t)$ is a mapping from state-action pairs to the expected future return when acting under a particular policy π.

    Intuition: "How good is this specific action here, if I continue behaving like my policy π?"

  - `V-function vs. Q-function:` Bei der V-function weiß ich nur, wie gut es ist, in einem bestimmten Zustand zu sein, wenn ich nach meiner Policy π agiere, aber nicht, wie gut eine konkrete Aktion in diesem Zustand ist. Bei

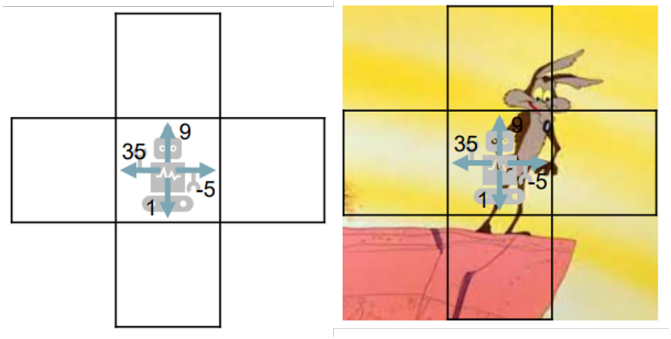


Figure 3.5: State-Action Value Function

der Q-function hingegen weiß ich bereits, wie gut die konkrete Aktion a_t im Zustand s_t ist, wenn ich weiterhin nach meiner Policy π agiere. Siehe dazu die Abbildung 3.4 und 3.5.

- `Dynamics model:` A Dynamic model $p(s_{t+1}|s_t, a_t)$ predicts the next state s_{t+1} given the current state s_t and action a_t. Video prediction models can be used as robot dynamics models. By learning to predict future video frames based on current observations and actions, the model can capture the underlying dynamics of the environment. This allows the robot to anticipate the consequences of its actions and make informed decisions based on predicted future states.
- `Reward model:` A reward function $r(s_t, a_t)$ defines the reward received by the agent for taking action a_t in state s_t. It can be learned from video data by observing the outcomes of actions and inferring the underlying reward structure that motivates those actions. For example, if a video shows a robot successfully picking up an object, the reward function can learn to assign a positive reward to that action in similar states.

3.2.2 Imitation Learning

Imitation Learning (IL) or Learning from Demonstration is a general term for methods in which the agent derives or learns its behavior from expert demonstrations.

Imitation learning makes it possible to use "in the wild" videos—such as those found in great variety on the internet, for example on platforms like YouTube — to train AI models.

The demonstration videos can show a robot, a human, or even a loose sequence of actions, both in the real world or in a simulation environment, performing a specific task. However, videos without a clearly defined objective or with unknown activity can also be used.

This allows models to be trained to capture the visual and semantic relationships, as well as the physical dynamics of the real world. This, in turn, is a key prerequisite for the development of a generalist policy. Imitation learning can be divided into Behavior Cloning and Learning from Videos/Observation.

3.2.2.1 Behavior Cloning (BC)

Behavior Cloning (BC) is a fundamental approach within imitation learning, where a policy is learned directly from **annotated expert demonstrations**. The training data consists of state–action pairs (s_t, a_t), and the problem is formulated as a standard supervised learning task.

Conceptually, Behavior Cloning can be understood as a form of direct imitation ("direct copying") of expert behavior, without explicitly modeling the underlying system dynamics or task objectives. Due to its simplicity and scalability, BC is one of the most widely used methods in robot learning from datasets.

BC is used to train Vision Language Action models and for pre-training models used in Reinforcement Learning, see Section **XXX**

Definition:

Behavior Cloning (BC) learns a policy by supervised learning on expert demonstrations:

$$\pi_\theta(a_t \mid s_t) \tag{3.2}$$

where:

- s_t: State / Observation (images, proprioception, etc.) at time t
- a_t: Expert action at time t

The Policy $\pi_\theta(a \mid s)$ is optimized by minimizing the prediction error between the actions generated by the model and the corresponding expert actions.

$$\min_\theta \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\mathcal{L}(\pi_\theta(s), a)\right] \tag{3.3}$$

where:

- $\pi_\theta(s)$: The action your robot's policy predicts given state s.
- a: The actual action the expert took in that same state.
- $\mathcal{L}$: Usually Mean Squared Error (MSE) for continuous control or Cross-Entropy for discrete actions.
- $\mathbb{E}_{(s,a)\sim\mathcal{D}}[\cdot]$: Take all the state-action pairs (s, a) inside the expert dataset $\mathcal{D}$, and calculate the average value of the expression inside the brackets.

3.2.2.2 Learning from Videos / Observation

Learning from Videos (LfV), also referred to as Learning from Observation (LfO), is an Imitation Learning approach that relies solely on **video demonstrations without explicit action annotations**. In contrast to Behavior Cloning, where state–action pairs are directly available, LfV aims to infer the underlying actions or high-level intent purely from perceptual data, i.e., mapping observations to actions $(s \to a)$. It often uses *Representation Learning*, to learn and discover the most important (unknown)

underlying data characteristics (features) by transforming and reducing input in a fully data-driven and automatic manner.

A central challenge in LfV is the **inference problem**: determining which actions correspond to the observed behavior in the video. Since no action labels are provided, additional methods and techniques are required to recover or approximate the missing action information. See chapter **XXX** for more information.

Due to this property, LfV is typically categorized as a **weakly supervised or self-supervised imitation learning** approach. Its main advantage lies in scalability, as it enables leveraging large amounts of unannotated video data, including internet videos or human demonstrations, without the need for costly data annotation. Furthermore, additional actions can be derived from annotated video data too.

In robotics, Learning from Videos is commonly used as a **pretraining step**. The learned representations capture visual, semantic, and dynamical aspects of the environment, which can later be refined using methods such as Behavior Cloning with annotated data or in Reinforcement Learning. This combination allows models to first learn the real worlds dynamics from large-scale video data and subsequently specialize on task-specific robot demonstrations.

Definition:

Learning from Videos (LfV) learns to infer actions from observations in a weakly supervised or self-supervised manner.

$$\hat{a}_t = f(s_t, s_{t+1}) \tag{3.4}$$

where:

- $\hat{a}_t$: The "latent" or predicted action.
- s_t: The current state/video frame.
- s_{t+1}: The next state/video frame.
- f: The inverse dynamics function (often a neural network) that maps state transitions to actions.

Chapter 4

Imitation Learning fundamentals

4.1 The Data Problem in Robotics

4.2 Why Imitation Learning?

4.2.1 Idea behind Imitation Learning:

When we want to learn a new skill, we often turn to How-To videos. We learn the new skill through speech instructions and visual observations in the video.

`Advantages of Imitation Learning:`

Visual IL ...

- is ideal for teaching tasks that are difficult or impossible to describe in words.
- allows robots to infer tasks from expert demonstrations in different environments than the robot.
- can be more data efficient than reinforcement learning, especially in high-dimensional state spaces, like a complex robot environment.
- can leverage large datasets of human demonstrations, which are often easier to collect than expert demonstrations in the specific domain.

`Problems in RL:` A Task in RL is commonly defined through a reward function, which assigns a high reward to desirable behavior and a low reward to undesirable behavior. However, designing good reward functions is challenging for every task, and many methods based on computer vision rely on annotated demonstrations in every new environment.

IL allows us to learn complex behaviors that are difficult to specify through a reward function, or it can help us to define a good adaptive reward function for RL.

Robotic specific problems: Gap between videos of humans and robots performing similar tasks Gap between simulation and Real-World Simulations & Trial and Error in

real World possible Limitations: time and computational consuming, Safety?

4.2.2 Instruct robots

This section discusses the various ways in which the robot can be taught its task and how the interaction between humans and robots can be designed.

Scene Understanding / Intent Inference The robot derives its task solely from the environment and the objects present in it [5].

`One Policy, Multiple Tasks:` In this setting, a single policy is typically specialized on a finite set of task instances, as they must be explicitly trained for those tasks. Tasks that are not represented or underrepresented in the training dataset, can therefore often not be executed reliably [2, 7]. For example, if the policy has been trained on multiple manipulation tasks and one of them involves pouring liquid from a teapot into a cup, the policy will execute this behavior when a matching scene is observed during inference [5]. However, generalization to other objects or situations is often limited. Using different teapots or cups may not be problematic as long as the relevant visual features are recognized. In contrast, performance typically decreases under substantial domain shift (significantly different objects or scenes) [5]. If a corresponding pouring behavior from a pot was not included during training, the policy may fail to transfer the action to that new object category [2, 5]. A major limitation of this approach is task ambiguity caused by overlapping object context across different behaviors. If the same object types, such as a teapot and a cup, appear in both a pick-and-place task and a pouring task, the policy may not be able to disambiguate the correct action target based on scene context alone [13].

`One Policy, One Task:` An alternative approach is to train a separate policy for each individual task, which is then selected explicitly during inference. The task-specific policy selection can be performed either manually by a human or automatically by a separate AI classification model that analyzes the scene and determines the appropriate task-specific policy [3]. For example, one policy could be trained for "pouring", another for "opening a drawer", and another for "setting up an overturned object". This modular design is closely related to task-aware expert routing and decoupled planning-and-execution frameworks in robotic manipulation [3].

This decoupling enables training each policy on significantly larger and more diverse datasets containing varied object appearances and environmental conditions, while keeping the core task semantics fixed [10]. Consequently, these specialized policies often support more robust generalization across novel object instances and environmental variations compared to the `One Policy, Multiple Tasks` paradigm. [1, 9, 13]

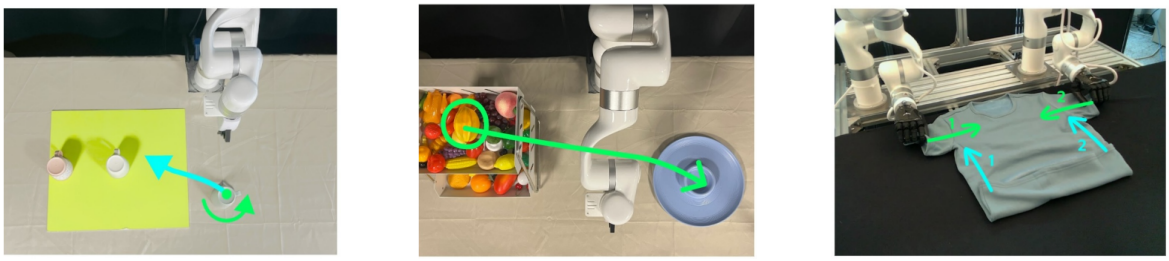
Language instructions: For humans is the most comfortable and natural way to instruct the robot with voice commands. This is very useful for simple tasks that are easy to describe. But complex tasks, where execution must follow exact instructions, are sometimes difficult to describe in words. For example, how would you teach a humanoid robot to decorate tables exactly like a sample table, especially when

various decorations need to be assembled in a complex arrangement? Furthermore, ambiguities may arise, and the same instruction may be interpreted differently in different linguistic contexts. For example, the word "open" is used for: open drawer, open cabinet, open container with lid or open jar with screw cap. Although the same verb is used here, each interaction requires a completely different motor control.

A big advantage to command the robot with voice commands is that it enables an interactive dialogue between humans and the robot. If the robot does not understand the instructions, it can ask follow-up questions to clarify any misunderstandings. This helps minimize the problems associated with voice commands.

Image instructions: Robots can also be instructed using visual inputs such as images. Compared to language, image-based instructions provide a more concrete and unambiguous specification of spatial arrangements, object relations, and desired outcomes.

A common approach is the use of *goal images*, where the robot is given a visual representation of the desired final state. In this setting, the task is formulated as a goal-conditioned problem: the robot observes the current state and acts such that it minimizes the difference to the goal image. This is often referred to as goal-conditioned behavior. However, this approach does not explicitly specify *how* the task should be executed, only *what* the final configuration should look like. But some works have addressed this issue with the usage of annotations in the goal images to specify the task execution. For example [6] highlights the relevant objects and regions with circles and arrows to visualize the task execution in the image [1].



(a) Task: Rotate cup and place it on table

(b) Task: Put fruit into bowl

(c) Task: Fold T-shirt

Figure 4.1: Image-based instructions for task execution [1].

To address this limitation, *subgoal images* or intermediate states can be provided. These guide the robot through a sequence of visually defined steps, effectively decomposing complex tasks into simpler stages. This is particularly useful for long-horizon or multi-step manipulation tasks.

A key advantage of image instructions is their ability to capture fine-grained spatial details that are difficult to describe in language, such as precise object placement, orientation or symmetry alignments. However, they also introduce challenges related to perception, such as viewpoint variation, occlusions of objects in the real environment, or the use of in-the-wild goal-images with varying lighting and background conditions.

Video-Based Instructions: Demonstration videos are a powerful way to instruct robots,

as they contain rich visual and temporal information. In contrast to single images, videos provide not only the desired goal state of a task but also the sequence of intermediate steps required to reach that goal.

In this setting, the task is typically formulated as an imitation learning problem, where the robot learns a policy that maps visual observations to actions by imitating the demonstrated trajectory. The temporal structure of the video enables the robot to infer *how* a task should be executed, rather than only *what* the final outcome should look like. Depending on the setup, this may also require inferring latent actions from visual observations alone.

Ideally, the robot model should be able to understand it's task from in-the-wild demonstration videos of both robots and humans. In-the-wild videos refer to video data collected in unconstrained, real-world environments, without controlled settings, standardized viewpoints, or curated conditions. Such data exhibits large variability in lighting, backgrounds, and camera motion, making it significantly more challenging for perception and learning compared to controlled laboratory data.

`Human Video Demonstrations:` In this paradigm, the robot learns from videos of humans performing a task. This is particularly appealing, as large amounts of human demonstration data are readily available (e.g. online videos). However, a key challenge is the embodiment gap: human motions and kinematics differ significantly from those of robots. Therefore, the robot must learn to map observed human actions to its own action space.

A key challenge of human video demonstrations is that they do not necessarily represent optimal task executions. Demonstrations often reflect individual preferences, inefficiencies, or context-specific behaviors, resulting in suboptimal trajectories. In many cases, a video captures only a single possible way of solving the task, without providing information about alternative or more efficient strategies. However, the objective for the robot is typically to perform tasks efficiently and robustly. Therefore, the learned policy must be able to generalize beyond the demonstrated behavior and, if necessary, improve upon it, rather than directly imitating potentially suboptimal actions.

`Robot Video Demonstrations:` The demonstrations are performed by the robot itself or by another robot with similar embodiment. This avoids the human-robot embodiment gap and provides more directly transferable data. However, collecting such demonstrations is often expensive and time-consuming, as it requires access to robotic systems and typically controlled environments. In addition, purely visual robot demonstrations may still lack explicit action labels.

This means that the robot must still infer the underlying actions and intentions from the visual data. Despite these challenges, robot video demonstrations can provide valuable data for learning, as they are more closely aligned with the robot's own capabilities and constraints compared to human demonstrations.

`Teleoperation:` A common method to generate high-quality robot demonstrations is teleoperation, where a human operator directly controls the robot, e.g., via a joystick or

a dedicated control interface. In contrast to passive video demonstrations, teleoperation provides access to both the visual observations and the corresponding action and trajectory data.

As a result, the demonstrated actions are already expressed in the robot's action space, which significantly simplifies reproducing the expected behavior. Such demonstrations are particularly important for fine-grained manipulation tasks that require precise control and accurate timing. However, teleoperation may require specialized hardware and does not scale as easily as collecting passive video data.

Chapter 5

5

Bibliography

[1] Yongjie Bai et al. *Learning to See and Act: Task-Aware View Planning for Robotic Manipulation*. Version 3. Oct. 28, 2025. DOI: 10.48550/arXiv.2508.05186. arXiv: 2508.05186 [cs]. URL: http://arxiv.org/abs/2508.05186 (visited on 05/02/2026). Pre-published.

[2] *Boosting Robotic Manipulation Generalization with Minimal Costly Data*. URL: https://arxiv.org/html/2503.19516v2 (visited on 05/02/2026).

[3] Yiteng Chen et al. *RoboRouter: Training-Free Policy Routing for Robotic Manipulation*. Mar. 12, 2026. DOI: 10.48550/arXiv.2603.07892. arXiv: 2603.07892 [cs]. URL: http://arxiv.org/abs/2603.07892 (visited on 05/02/2026). Pre-published.

[4] Rohan Jagtap. *Understanding the Markov Decision Process (MDP)*. Built In. URL: https://builtin.com/machine-learning/markov-decision-process (visited on 04/28/2026).

[5] Eric Jang et al. *BC-Z: Zero-Shot Task Generalization with Robotic Imitation Learning*. Feb. 4, 2022. DOI: 10.48550/arXiv.2202.02005. arXiv: 2202.02005 [cs]. URL: http://arxiv.org/abs/2202.02005 (visited on 05/02/2026). Pre-published.

[6] Yanbang Li et al. "Robotic Visual Instruction". In: (2025).

[7] Zhao Mandi et al. *CACTI: A Framework for Scalable Multi-Task Multi-Scene Visual Imitation Learning*. Feb. 16, 2023. DOI: 10.48550/arXiv.2212.05711. arXiv: 2212.05711 [cs]. URL: http://arxiv.org/abs/2212.05711 (visited on 05/02/2026). Pre-published.

[8] Tim Miller. *Markov Decision Processes — Mastering Reinforcement Learning*. URL: https://gibberblot.github.io/rl-notes/single-agent/MDPs.html (visited on 04/28/2026).

[9] *Multitask Robot Manipulation Policies*. URL: https://www.emergentmind.com/topics/multitask-robot-manipulation-policies (visited on 05/02/2026).

[10] Antonin Raffin et al. "DECOUPLING FEATURE EXTRACTION FROM POLICY LEARNING: ASSESSING BENEFITS OF STATE REPRESENTA- TION LEARNING IN GOAL BASED ROBOTICS". In: (2019).

[11] Coursera Staff. *What Is a Markov Decision Process?* Coursera. May 15, 2025. URL: https://www.coursera.org/articles/what-is-a-markov-decision-process (visited on 04/28/2026).

[12] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning: an introduction*. Adaptive computation and machine learning. Cambridge, Mass: MIT Press, 1998. 322 pp. ISBN: 978-0-262-19398-6.

[13] Aviv Tamar et al. "IMITATION LEARNING FROM VISUAL DATA WITH MULTIPLE INTENTIONS". In: (2018).

[14] Walkerastro. *Action Space, State Space, Observation Space demystified*. Medium. Aug. 26, 2024. URL: https://medium.com/@walkerastro41/action-space-state-space-observation-space-demystified-6c9c00a355b4 (visited on 04/28/2026).

List of Figures

3.1 Base concept of Reinforcement Learning 7
3.2 Overview Robotic Learning 8
3.3 Training Progress in Deep Learning and Reinforcement Learning 10
3.4 State-Value Function 11
3.5 State-Action Value Function 12

4.1 Image-based instructions for task execution [1]. 17

List of Tables